Sorting

- → Why Sorting ?
- → Min cost to remove all element
- → Nobel Integer → ver1, ver2
- → Comparator
- → Count digit sort

sorting : Arranging data in inc / dec order based on parameter

Ex 1 : 3, 8, 9, 14, 19 → based on value asc

Ex2 : 19, 14, 11, 8, 6 → desc

Ex3 :	1	5	3	9	6	10	12
# factors	1	2	2	3	4	4	6

⇒ sort()

To sort an array of N elements

TC : $O(n \log n)$ ✓

Q ⇒ Min Cost to remove all Element.

Given N array element, at every step remove an array element. Cost to remove element = Sum of array element present in array

Ex1: {2, 1, 4}

Remove 2 : {2, 1, 4} = 2+1+4 = 7

Remove 1 : {1, 4} = 1+4 = 5

Remove 4 : {4} = 4

16

Remove 4 : {2, 1, 4} = 7

Remove 2 : {1, 2} = 3

Remove 1 : {1} = 1

11

Quiz 1 : {4, 6, 1}

{4, 6, 1} 6 : 4+6+1 = 11

{4, 1} 4 : 4+1 = 5

{1} 1 : 1 = 1

17

Quiz 2 : {3, 5, 1, -3}

5 : 3+5+1+(-3) = 6

3 : 3+1+(-3) = 1

1 : 1+(-3) = -2

-3 : (-3) = -3

2

$\{a, b, c, d\}$

Remove a : $\{a, b, c, d\}$

Remove b : $\{b, c, d\}$

Remove c : $\{c, d\}$

Remove d : $\{d\}$

Total cost

Cost

$a+b+c+d$

$b+c+d$

$c+d$

d

$a \cdot 1 + 2b + 3c + 4d$

1^{st} Max

2^{nd} Max

3^{rd} Max

$\overset{0}{a}, \overset{1}{b}, \overset{2}{c}, \overset{3}{d}$

$a*(0+1)$

$b*(1+1)$

$c*(2+1)$

Sort Array in Desc order

```
sum = 0
i=0;  i < N; i++
      sum = sum + arr[i] * (i+1);
```

$O(n \log n + n)$

TC: $O(n \log n)$

Q => Nobel Integers

Given N array elements, calculate no. of nobel integers present in it.

a[i] is said to be Nobel int

{ No. of elements < arr[i] } = arr[i]

Ex1: { 1, -5, 3, 5, -10, 4 }

2 1 3 5 0 4

Ex2: { -3, 0, 2, 5 }

0 1 2 3

Ex3: { -10, -5, 1, 3, 4, 5, 10 }

0 1 2 3 4 5 6

```
cnt = 0
i=0 ; i<N ; i++
    int less = 0
    j= 0 , j<N ; j++
        if (arr[j] < arr[i])
            less ++
    if (less == arr[i])
        cnt ++;
```

$O(N^2)$

Solu2 :

sort & compare arr[i] = = i

sort the array ✓

```
ans = 0
i = 0 ; i < N; i++
    if (arr[i] == i)
        ans++
return ans;
```

O(n log n)

version 2

Ex: {0, 2, 2, 4, 4, 6}
↓ 0 1 1 3 3 5

Quiz : [-10, 1, 1, 1, 4, 4, 4, 7, 10]
0 1 1 1 4 4 4 7 8

Quiz : [-10, 1, 1, 2, 4, 4, 4, 8, 10]
0 1 1 3 4 4 4 7 8

Quiz : [-3, 0, 2, 2, 5, 5, 5, 5, 8, 8, 10, 10, 10, 14]
0 1 2 3 4 5 6 7 8 9 10 11 12 13

Ceis — 0 1 2 2 4 4 4 4 8 8 10 10 10 13

obs 1: If elem is same as prev
then count wont change

obs 2: If ele comes first time
no. of elem less than a[i] = i

sort the array
int ans = 0
in less = 0
if (arr[0] == 0) ans++;
i = 0 ; i < N; i++
if (arr[i] != arr[i-1])
less = i
else { // arr[i] is repeating
// less won't change
}
if (less >= arr[i]);
ans++;

O(n log n)

Break: 8:31

Comparator } → prog Language

Q ⇒ Given N array elements, sort them in increasing order of {no. of factors}

If 2 elem have same no. of factors, element with less value should come first

{ 9, 3, 4, 8, 16, 37, 6, 13, 15 }

#factors { 3, 2, 3, 4, 5, 2, 4, 2, 4 }

3, 13, 37, 4, 9, 6, 8, 15, 16

Ex1 :

2 5 → 3   16 → 5

Ex 2 :

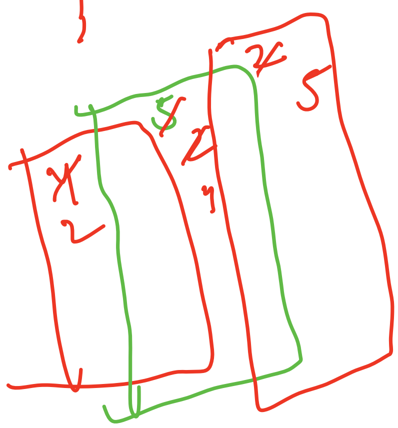
10 → 4   9 → 3

Arrays . sort (arr, comp)

bool comp(int a, int b)

{

```
int f1 = factor(a);
int f2 = factor(b);
if (f1 < f2)
    return true;
else if (f1 == f2 && a < b)
    return true;
else
    return false;
}
```



Q => Given N array elements, sort them in increasing order of No of digits. If 2 elements have same no. of digits element with more value should come first

93, 2, 37, 639, 8, 100, 345, 49

2, 1, 2, 3, 1, 3, 3, 2

o/p: 8, 2, 93, 49, 37, 639, 345, 100

```
bool   comp( int  a,   int  b)
{
    // if we want  a to come  before b
    // return  true
    int d1= countDigits (a);
    int d2 = countDigits (b);
    if (d1 < d2)  return true;
    else if (d1 ==d2  && a >b)
        return  true;
    else
        return false;
}
```

Doubts:

1, 2, 3, 3

ele1 : 1 , 3

count : ~~1~~ ~~0~~ ~~1~~ 2

```
i=0 ; i <N ; i++
    if ( freq1 ==0 && arr[i] != ele2)
        { ele1 = arr[i]
          freq1 = 1.
        }
    else if (arr[i] == ele1)
```

```
        freq1++;
else if ( freq2==0)
    {   ele2 = arr[i]
        freq2 = 1;
    }
else if ( arr[i] == ele2)
        freq2++;
else
    {
        freq1--;
        freq2--;
    }

c1 = 0 , c2 = 0
i=0 ; i<N ; i++
    if ( arr[i] == ele1) c1++;
    elseif ( arr[i] == ele2) c2++
```